BACHELOR 3 · CYBERSÉCURITÉ · ADMINISTRATION SYSTÈMES LINUX / WINDOWS

InfraReady.

Dossier technique final

Déployer. Automatiser. Superviser. Diagnostiquer.

3	8	4	131	44
machines virtuelles	playbooks Ansible	cibles supervisées	preuves versionnées	commits documentés

Laboratoire	QEMU/KVM sur Arch Linux · réseau libvirt 192.168.50.0/24 · domaine `infraready.test`
Machines	● **WIN-DC01** ● **LINUX01** ● **OPS01**
Dépôt	`github.com/thom972/infraready` · branche `Keo`
État constaté	10 novembre 2026 · séances 1 à 8

Projet fil rouge 2026. Ce dossier est rédigé à partir de la documentation du dépôt (`docs/`) et décrit uniquement ce qui est démontré par une preuve versionnée (sortie de commande, capture, exécution). Aucun mot de passe, jeton ni clé n'apparaît dans ce document.

Sommaire

01 Contexte, besoin et périmètre 3
02 Architecture, adressage, réseau et DNS 4
03 Prérequis et procédures de déploiement 6
04 Administration Windows — Active Directory 8
05 Administration Linux — services, droits et journaux 11
06 Stockage et maintenance 13
07 Automatisation Ansible 15
08 Supervision — Prometheus et Grafana 17
09 GitHub et intégration continue 20
10 Sécurité et gestion des accès 22
11 Rapport d'incident — absence de métriques de LINUX01 24
12 Écarts, choix et évolutions 26
Annexe A — Correspondance avec les grilles 27
Annexe B — Commandes de référence 29
Annexe C — Index des preuves 30

Synthèse : exigences du sujet et état réel

Partie	Exigence	État	Preuve principale
A	Installation des 3 VM, réseau, DNS	Validé	Ping et résolution entre VM, zone DNS (§02)
B	Administration Windows : AD DS, comptes, groupes, droits, journaux	Validé	OU, 2 groupes, 4 comptes, partage testé, journaux relus (§04)
C	Administration Linux : services, droits, stockage, journaux	Validé	Nginx, setgid 2775, ext4 persistant (§05–06)
D	Automatisation Ansible Linux et Windows	Validé	8 playbooks, 2e passage `changed=0` (§07)
E	Maintenance Linux et Windows	Validé	`maintenance.yml`, 4 KB Windows installées (§06)
F	Supervision, tableau de bord, alerte	Validé	4 cibles UP, alerte disque pending → firing (§08)
G	GitHub et intégration continue	Validé	Workflow de 14 contrôles sur push et PR, exécutions réussies (§09)
H	Diagnostic d'incident	Validé	Rapport complet, retour à la normale prouvé (§11)

Comment lire ce dossier
Chaque section correspond à un critère des grilles d'évaluation, indiqué en tête de section. L'annexe A reprend tous les sous-critères des grilles pairs et formateur, avec la section et la preuve qui y répondent. La traçabilité exigence par exigence est dans `docs/tracabilite.md`.

01

Contexte, besoin et périmètre

InfraReady est une entreprise fictive qui exploite des serveurs Windows et Linux. Ses configurations sont faites à la main, peu documentées et difficiles à reproduire ; les incidents sont détectés tard. Le projet construit un laboratoire qui répond à ces quatre besoins :

Reproduire
Toute configuration est décrite dans un playbook versionné et rejouable sans effet de bord.

Sécuriser
Moindre privilège, secrets chiffrés, flux réseau limités au strict nécessaire.

Superviser
Métriques CPU, mémoire, disque et disponibilité des trois serveurs, avec alertes.

Diagnostiquer
Méthode écrite : symptôme, tests, journaux, cause, correction, vérification.

Périmètre et règles du laboratoire

- Toutes les opérations restent dans le réseau privé 192.168.50.0/24 du poste hôte. Aucun système extérieur n'est scanné ni testé.
- Les machines sont des VM jetables : chaque changement majeur est précédé d'un snapshot qcow2.
- Le dépôt Git est la source unique : il est cloné sur OPS01 et mis à jour par `git pull` après chaque modification.
- Règle absolue du dépôt : aucun mot de passe, jeton, clé privée, ISO ni disque virtuel.

Choix techniques principaux

Domaine	Choix	Justification
Hyperviseur	QEMU/KVM + virt-manager (E-01)	Natif sur l'hôte Linux, libre, snapshots qcow2, réseaux isolés
Réseau	Réseau libvirt NAT dédié (E-03)	Mises à jour possibles, services non publiés sur le LAN
Linux	Ubuntu Server 26.04.1 LTS (E-12)	Version disponible au moment de l'installation
Windows	Windows Server 2025, AD DS + DNS	Exigé par le sujet
Automatisation	Ansible core 2.20, inventaire YAML (E-11)	SSH pour Linux, WinRM pour Windows, Vault pour les secrets
Supervision	Prometheus + Grafana + exporters	Modèle **pull**, tableau de bord et alertes provisionnés par code

02

Architecture, adressage, réseau et DNS

Critère 1 · 15 pts Infrastructure, réseau et DNS

Couvre : rôles des machines · plan d'adressage · connectivité · DNS · schéma

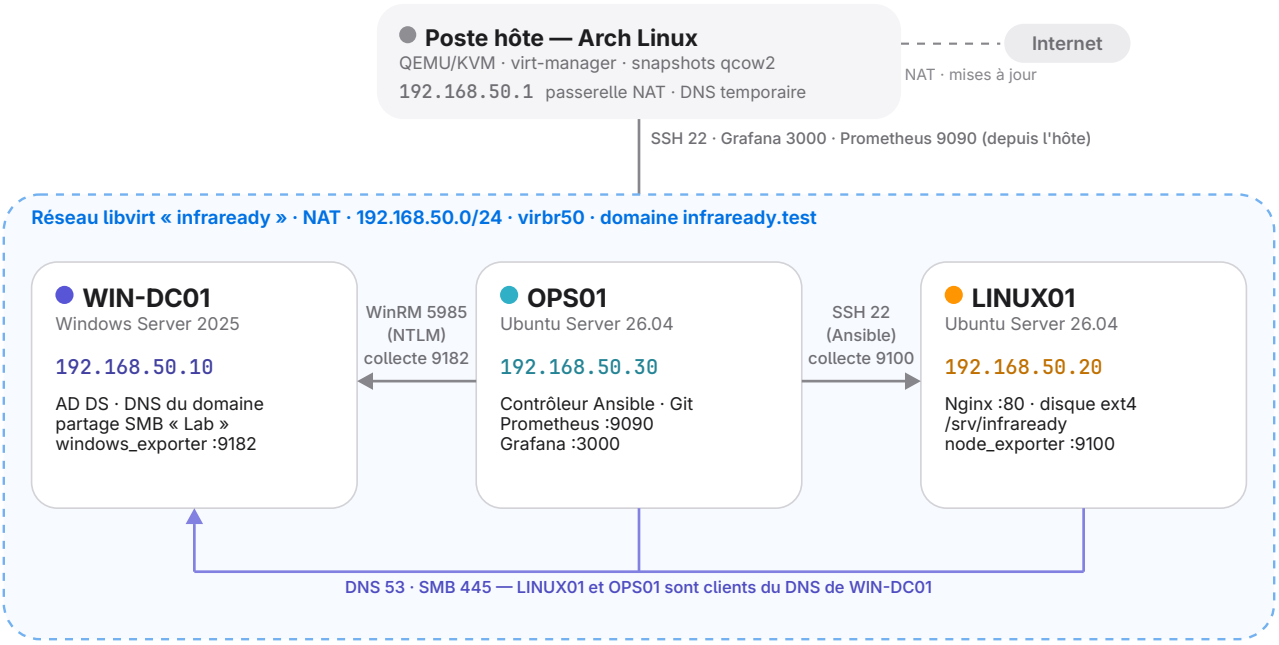


Figure 1. – Architecture déployée. Source modifiable : `docs/schemas/infraready-architecture.drawio`.

Rôle des machines

Machine	Système	vCPU / RAM / disque	Rôle
WIN-DC01	Windows Server 2025 Standard	2 / 4 Go / 50 Go	Contrôleur du domaine `infraready.test`, DNS, partage SMB `Lab`, windows_exporter
LINUX01	Ubuntu Server 26.04.1 LTS	2 / 2 Go / 30 Go + 5 Gio	Service web Nginx, volume de données `/srv/infraready`, node_exporter
OPS01	Ubuntu Server 26.04.1 LTS	2 / 4 Go / 50 Go	Contrôleur Ansible, clone Git, Prometheus, Grafana

Aucune fusion de rôles : les trois VM du sujet sont présentes. Les ressources tiennent sur le poste hôte ; seul le volume racine d'OPS01 a été agrandi à chaud (10 → 18,22 Gio, §06).

Plan d'adressage IP

Adresse	Usage	Type
`192.168.50.1`	Hôte : passerelle NAT, DNS temporaire	libvirt
`192.168.50.10`	WIN-DC01	fixe
`192.168.50.20`	LINUX01	fixe
`192.168.50.30`	OPS01	fixe
`.100 – .199`	DHCP, installations uniquement	dynamique

Prévention des conflits. Le réseau dispose de son propre pont `virbr50`. La plage DHCP est volontairement placée hors des adresses fixes, qui sont réservées dans le plan (`docs/adressage-ip.md`) et déclarées en statique sur chaque VM (netplan, IP Windows).

DNS. Avant la promotion d'AD DS, les VM utilisaient le dnsmasq de libvirt (192.168.50.1, E-04). Elles utilisent ensuite 192.168.50.10, avec le suffixe `infraready.test`.

Configuration réseau versionnée

infra/libvirt/infraready-network.xml

```xml
<network>
  <name>infraready</name>
  <forward mode='nat'/>
  <bridge name='virbr50' stp='on'
delay='0'/>
  <ip address='192.168.50.1'
      netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.50.100'
             end='192.168.50.199'/>
    </dhcp>
  </ip>
</network>
```

configs/linux01/netplan/01-lab.yaml

```yaml
network:
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false
      addresses: [192.168.50.20/24]
      routes:
        - to: default
          via: 192.168.50.1
      nameservers:
        addresses: [192.168.50.10]
        search: [infraready.test]
```

Connectivité et résolution DNS testées

S04_dns-clients-linux.txt — depuis LINUX01 et OPS01

```
=== linux01 ===
DNS Domain: infraready.test
192.168.50.10  WIN-DC01.infraready.test
192.168.50.30  ops01.infraready.test
rtt min/avg/max = 0.637/0.835/1.033 ms
=== ops01 ===
192.168.50.10  WIN-DC01.infraready.test
192.168.50.20  linux01.infraready.test
Link 2 (enp1s0): 192.168.50.10
```

S04_win-dc01_zone-dns.txt — zone infraready.test

```
HostName        Adresse
--------        -------
@               192.168.50.10
LINUX01         192.168.50.20
OPS01           192.168.50.30
win-dc01        192.168.50.10

20.50.168.192.in-addr.arpa
          -> LINUX01.infraready.test
```

La zone directe contient les trois serveurs ; la zone inverse résout LINUX01. Les tests de réception de la séance 2 (`S02_*_reception.txt`) vérifient l'IP, la route par défaut, le ping de la passerelle et des deux autres VM (3/3 reçus) et la résolution d'un nom Internet.

Un échec DNS réel, et son interprétation

Lors de la réception (séance 2), `getent hosts WIN-DC01.infraready.test` renvoyait le **code 2** (nom introuvable) sur LINUX01 et OPS01, alors que `ubuntu.com` se résolvait. Les VM interrogeaient encore le DNS temporaire de libvirt (192.168.50.1, E-04), qui ne connaît pas la zone `infraready.test`. Après la promotion d'AD DS et le passage du DNS à 192.168.50.10, le même nom se résout (`S04_dns-clients-linux.txt`). Depuis la séance 8, `securite.yml` refait ce test à chaque passage et échoue si la réponse ne contient pas 192.168.50.10.

03

Prérequis et procédures de déploiement

Critère 3 · 15 pts Automatisation Ansible et reproductibilité

Couvre : commandes · prérequis · méthode de rejeu par un autre administrateur

Le laboratoire se reconstruit à partir du dépôt seul : chaque configuration est un playbook, chaque secret est dans un coffre chiffré. Source : `README.md` et `docs/ansible.md`.

Prérequis

Où	Élément	Installation / vérification
Poste hôte	QEMU/KVM, libvirt, virt-manager ; réseau `infraready`	`virsh net-define infra/libvirt/infraready-network.xml` `net-start`, `net-autostart`
OPS01	Ansible core 2.20 ou plus, `python3-winrm`	`sudo apt install ansible python3-winrm`
OPS01	Collections `ansible.windows`, `community.windows`, `microsoft.ad`, `community.general`, `ansible.posix`	`ansible-galaxy collection install -r collections/requirements.yml`
OPS01	Clé SSH d'automatisation	`~/.ssh/id_ed25519_ansible`, droits 600, phrase de passe ; clé publique chez `ansible@LINUX01`
OPS01	Mot de passe du coffre, mot de passe sudo de `keo`	Gestionnaire de mots de passe du binôme, jamais dans le dépôt
WIN-DC01	WinRM HTTP 5985 accepté depuis OPS01 seulement	`scripts/windows/Set-WinRMLabScope.ps1`

Trois coffres Ansible Vault

Coffre	Contenu (noms des variables seulement)	Utilisé par
`group_vars/linux/vault.yml`	`ansible_become_password` du compte `ansible`	sudo sur LINUX01
`group_vars/ops/vault.yml`	`grafana_admin_password`	`supervision.yml`
`group_vars/windows/vault.yml`	mot de passe de `svc-ansible`, `lab_user_initial_password`	WinRM, `windows.yml`

Ansible charge le coffre d'un groupe dès qu'un de ses hôtes est visé : **toute commande qui touche une VM demande `--ask-vault-pass`**. `-K` (sudo de `keo`) s'ajoute dès qu'OPS01 est visé.

Procédure standard pour chaque playbook

1 Syntaxe `--syntax-check`

2 Cibles `--list-hosts`

3 Simulation `--check --diff`

4 Exécution RECAP : `failed=0`, `unreachable=0`

5 Rejeu second passage : `changed=0`

Le dépôt est l'unique source : les modifications sont faites sur le poste du binôme, poussées sur GitHub, puis récupérées sur OPS01 par `git pull`. Chaque modification est d'abord validée par le workflow GitHub Actions (§09).

Déployer un labo neuf

OPS01 — ~/infraready

```
git pull
ansible-galaxy collection install \
    -r collections/requirements.yml
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_ansible
ansible-inventory --graph
ansible linux -m ping --ask-vault-pass
ansible windows -m ansible.windows.win_ping \
    --ask-vault-pass
```

Ordre de déploiement (README.md)

```
ansible-playbook playbooks/windows.yml \
    --ask-vault-pass
ansible-playbook playbooks/linux.yml \
    --ask-vault-pass
ansible-playbook playbooks/supervision.yml \
    -K --ask-vault-pass
ansible-playbook playbooks/securite.yml \
    -K --ask-vault-pass
ansible-playbook playbooks/verification.yml \
    -K --ask-vault-pass
```

04

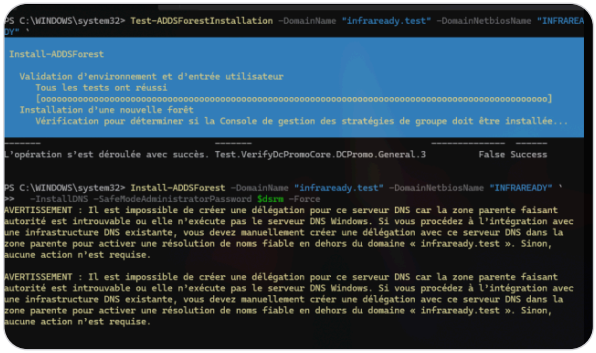
Administration Windows — Active Directory

Critère 2 · 20 pts Administration Windows et Linux

Couvre : AD DS et DNS · comptes, groupes et droits · journaux · tests

Domaine et annuaire

WIN-DC01 a été promu contrôleur d'une nouvelle forêt `infraready.test` (`Test-ADDSForestInstallation` puis `Install-ADDSForest`), avec le rôle DNS intégré. L'annuaire, les groupes, les comptes de test et le partage sont ensuite créés par le playbook `playbooks/windows.yml`, et non à la main.



Capture 1. – Test puis promotion AD DS de WIN-DC01.

S04_win-dc01_ad-structure.txt

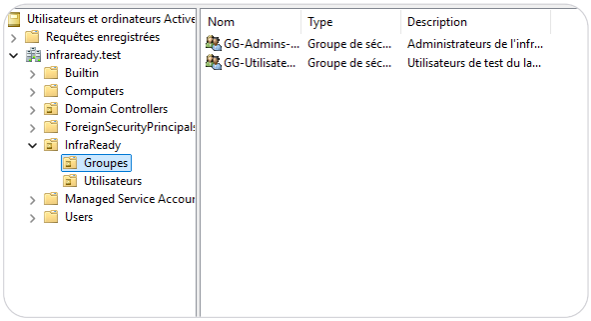
```
Name          DistinguishedName
InfraReady    OU=InfraReady,DC=infraready,DC=test
Utilisateurs  OU=Utilisateurs,OU=InfraReady,...
Groupes       OU=Groupes,OU=InfraReady,...

== GG-Admins-Infra ==
a.martin
== GG-Utilisateurs-Lab ==
a.martin
b.durand
c.petit
```

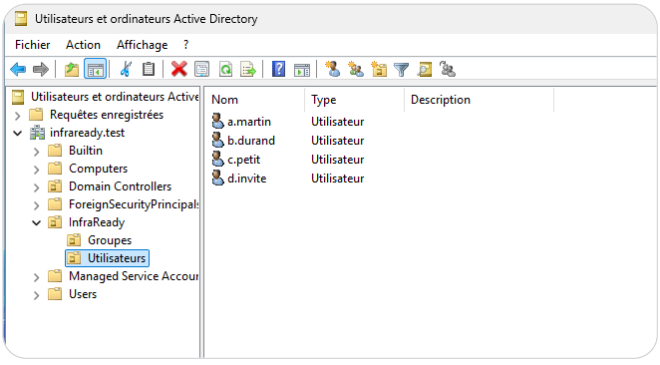
Comptes et groupes

Compte	Groupes	Usage
`a.martin`	GG-Admins-Infra, GG-Utilisateurs-Lab	Administrateur de l'infrastructure
`b.durand`, `c.petit`	GG-Utilisateurs-Lab	Utilisateurs standards
`d.invite`	aucun	Compte témoin pour prouver un accès refusé
`svc-ansible`	Admins du domaine (E-15)	Automatisation WinRM depuis OPS01 ; non délégable
`Administrateur`	Admins du domaine	Administration manuelle uniquement

Les deux groupes sont globaux, de type sécurité. **Les droits sont donnés aux groupes, jamais aux comptes** : ajouter ou retirer un utilisateur d'un groupe suffit à changer ses accès.



Capture 2. – Console « Utilisateurs et ordinateurs AD » : OU `InfraReady/Groupes`.



Capture 3. – OU `InfraReady/Utilisateurs` : les quatre comptes de test.

Droits sur le partage `\\WIN-DC01\Lab`

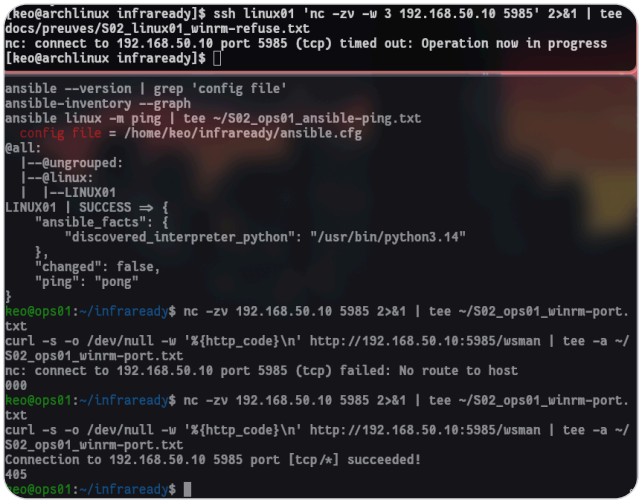
L'accès effectif est le plus restrictif des deux niveaux, partage **et** NTFS. L'héritage NTFS est coupé : seules les entrées listées s'appliquent.

Principal	Partage	NTFS
GG-Admins-Infra	Contrôle total	Contrôle total
GG-Utilisateurs-Lab	Modification	Modification
SYSTEM, Administrateurs	—	Contrôle total
Tout autre compte	Aucune	Aucun

Compte	Attendu	Obtenu (smbclient)
b.durand	écrit	fichier créé OK
a.martin	lit, supprime	listé, supprimé OK
d.invite	refusé	`NT_STATUS_ACCESS_DENIED` OK

Services, accès distant et journaux Windows

- **Services critiques surveillés** par `verification.yml` : NTDS, DNS, Netlogon, Kdc, WinRM, LanmanServer (tous `Running`, `S05_ops01_verification.txt`). Santé du contrôleur contrôlée par `dcdiag` après chaque maintenance.
- **WinRM** (TCP 5985, transport NTLM qui chiffre les échanges) n'est accepté que depuis OPS01 : la même requête depuis LINUX01 expire (capture ci-dessous).
- **Diagnostic du contrôleur** : `dcdiag` a réussi les tests Connectivity, Advertising, SysVolCheck, NetLogons, Services et FsmoCheck (`S04_win-dc01_dcdiag.txt`).



Capture 4. – WinRM : accepté depuis OPS01 (HTTP 405), refusé depuis LINUX01 (délai dépassé).

Contrôles Windows documentés (maintenance.md)

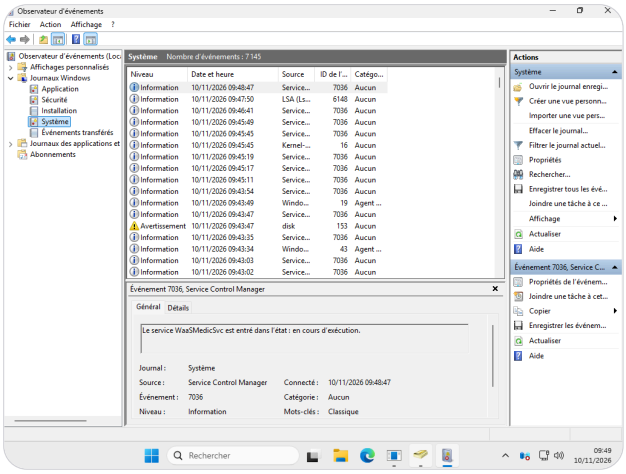
```
dcdiag /test:Services /test:Advertising `
  /test:NetLogons /test:SysVolCheck
Get-HotFix |
  Sort-Object InstalledOn -Descending |
  Select-Object -First 5
```

Lire les journaux (maintenance.md §5)

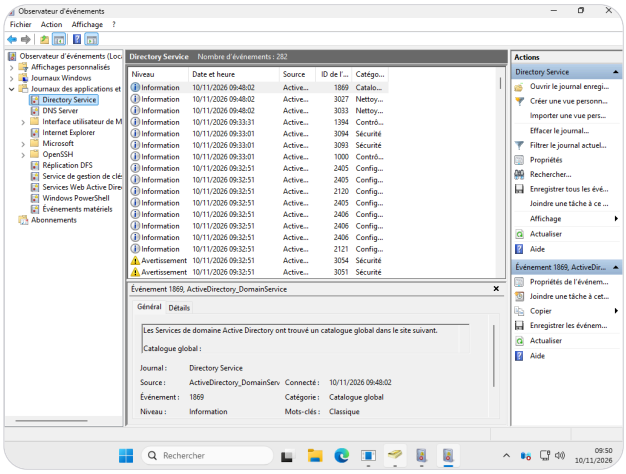
```
Get-WinEvent -LogName System -MaxEvents 50 |
  Where-Object Level -le 3
Get-WinEvent -LogName 'Directory Service' `
  -MaxEvents 50 | Where-Object Level -le 3
```

Journaux Windows relus

Les journaux de WIN-DC01 sont relus après chaque redémarrage et à chaque contrôle, dans l'Observateur d'événements (`eventvwr /c:System`) ou par `Get-WinEvent` (niveau ≤ 3 : critique, erreur, avertissement).



Capture 5. – Journal **Système** : 7 145 événements, démarrages de services (7036), un avertissement `disk` 153.



Capture 6. – Journal **Directory Service** : catalogue global trouvé (1869), avertissements de sécurité AD 3051 et 3054.

Relevé du 10/11/2026 : aucune erreur dans les événements affichés ; l'avertissement `disk` 153 signale une opération d'E/S relancée sur le disque virtuel, à surveiller s'il se répète.

05

Administration Linux — services, droits et journaux

Critère 2 · 20 pts Administration Windows et Linux

Couvre : services et processus · permissions · journaux · tests et validation

Comptes et privilèges

Machine	Compte	Authentification	Privilèges
LINUX01, OPS01	`keo` (nominatif)	Clé SSH, mot de passe en secours console	sudo avec mot de passe
LINUX01	`ansible`	Clé SSH **uniquement** (mot de passe SSH refusé)	sudo **avec** mot de passe issu d'Ansible Vault (E-14 corrigé en séance 8)
toutes	`root`	Aucune connexion SSH directe	—

Service applicatif : Nginx

Nginx est installé et activé au démarrage par `playbooks/nginx.yml`. Le script `scripts/linux/check-nginx.sh` vérifie son état et renvoie un code de sortie exploitable (0 si actif).

Vérifier, piloter, diagnostiquer

```
systemctl status nginx          # état, PID, dernières lignes
systemctl is-active nginx       # active / inactive
systemctl is-enabled nginx      # démarrage automatique
sudo systemctl restart nginx
sudo nginx -t                   # syntaxe de la conf
journalctl -u nginx --since -1h
ss -lntp | grep ':80'           # port en écoute
curl -I http://192.168.50.20    # réponse HTTP
ps -o pid,user,cmd -C nginx     # maître root, workers www-data
```

check-nginx.sh — avant / après le playbook (S03)

```
# avant
Unit nginx.service could not be found.
# après ansible-playbook playbooks/nginx.yml
OK : le service nginx est actif
code de sortie : 0
```

Le processus maître tourne en `root` pour ouvrir le port 80, les **workers** en `www-data` (`S05_linux01_nginx-processus.txt`). `verification.yml` contrôle en plus la réponse HTTP 200.

Permissions du volume de données

S04_linux01_stockage-verif.txt

```
drwxrwsr-x root infraready /srv/infraready
drwxrwsr-x root infraready /srv/infraready/
logs
```

Mode `2775` : propriétaire `root`, groupe `infraready` en écriture, lecture seule pour les autres. Le bit **setgid** (`s`) fait hériter le groupe `infraready` à tout fichier créé. Pas de `chmod 777`.

Test réel : `keo` (membre du groupe) crée puis supprime un fichier ; `ansible` (hors groupe) se voit refuser l'écriture.

Résultat testé et non supposé
Le test `test -w` renvoyait un faux négatif avec un groupe secondaire, alors que les droits étaient corrects. Le playbook vérifie donc par une **écriture réelle** (`touch` puis `rm`), qui ne peut pas mentir (`S04_linux01_diagnostic-droits.txt`).

Journaux Linux utiles

Commande	Ce qu'elle montre	Preuve
`journalctl -u ssh`	Connexions SSH acceptées par clé, ouvertures et fermetures de session	`S03_linux01_ssh-journal.txt`
`journalctl -u nginx`	Démarrages, arrêts et erreurs du service web	`S03_linux01_nginx-journal.txt`
`journalctl -u prometheus-node-exporter`	Utilisé pendant l'incident (§11) pour écarter une panne de l'exporter	`S07_linux01_diagnostic.txt`
`/var/log/ufw.log`	Connexions refusées par le pare-feu (journalisation `low`)	`docs/securite.md`

Un compte hors des groupes `adm` et `systemd-journal` ne voit que ses propres journaux (`S03_linux01_nginx-journal.txt`) : la lecture complète des journaux est elle-même un privilège.

06

Stockage et maintenance

Critère 4 · 10 pts Stockage et maintenance

Couvre : disque et montage · mises à jour Linux et Windows · contrôles · précautions

Disque de données de LINUX01

Élément	Valeur
Disque	qcow2 5 Gio, VirtIO, série `INFRA-DATA`
Désignation	`/dev/disk/by-id/virtio-INFRA-DATA` (jamais `/dev/vdX`)
Partition	`vdb1`, GPT, unique
Système de fichiers	ext4, étiquette `infradata`
Montage	`/srv/infraready`, `/etc/fstab` par UUID, `defaults,nofail`

Vérification de la persistance

```
$ lsblk -f /dev/vdb
└─vdb1 ext4 infradata bd3813fb-... 4,6G 0% /
srv/infraready
$ grep infraready /etc/fstab
UUID=bd3813fb-... /srv/infraready ext4
defaults,nofail 0 0
$ sudo findmnt --verify
$ sudo reboot   # puis :
$ findmnt /srv/infraready
/srv/infraready /dev/vdb1 ext4 rw,relatime
MONTAGE PERSISTANT OK
```

UUID plutôt que `/dev/vdb` : le nom d'un disque peut changer d'un démarrage à l'autre, pas l'UUID de son système de fichiers. **`nofail`** : si le disque manque, la VM démarre quand même au lieu de rester bloquée en mode urgence.

Garde-fous avant écriture (`playbooks/stockage.yml`)

```yaml
- name: Refuser de continuer si le disque est absent ou porte la racine
  ansible.builtin.assert:
    that:
      - disque.stat.exists
      - disque.stat.lnk_source is not search('vda')
      - "'vdb' not in racine.stdout"
    fail_msg: "Disque INFRA-DATA introuvable ou cible douteuse : arrêt sans rien modifier."
- name: Système de fichiers (jamais de reformatage si un système existe déjà)
  community.general.filesystem: { dev: "{{ data_part }}", fstype: ext4, force: false }
```

Avant le premier passage : snapshot `04-avant-disque-donnees` et sauvegarde automatique de `/etc/fstab` (`backup: true`).

Mises à jour

LINUX01 **OPS01** — Automatisé

`maintenance.yml` : vérification initiale → `apt upgrade: safe` (aucune suppression de paquet) **une machine à la fois** (`serial: 1`) → détection de `/var/run/reboot-required` → redémarrage seulement s'il est requis **et** autorisé (`-e autoriser_redemarrage=true`), jamais OPS01 → vérification finale.

WIN-DC01 — Recherche auto., install. manuelle

Ansible liste les mises à jour disponibles sans rien installer. Installation par Windows Update après snapshot, redémarrage en dernier, puis `Get-HotFix` et `dcdiag`. Un redémarrage du contrôleur coupe l'annuaire et le DNS : il reste une décision humaine.

Historique des maintenances

Date	Serveurs	Mises à jour	Résultat
22/10	LINUX01, OPS01	Paquets Ubuntu (mode safe), redémarrage non requis	`failed=0` ×3
22/10	WIN-DC01	KB5126646 (cumulative sécurité), KB5126645, KB5127690 (.NET), KB890830, en deux passages, snapshot préalable	`dcdiag` OK, 0 restante



Capture 7. – Historique Windows Update de WIN-DC01.

Contrôle de l'état des machines

`verification.yml` (lecture seule, aucun changement) se lance **avant et après** chaque maintenance :

Serveur	Services critiques	Volumes surveillés
LINUX01	ssh, nginx (+ HTTP 200)	`/`, `/boot`, `/srv/infraready`
OPS01	ssh	`/`, `/boot`
WIN-DC01	NTDS, DNS, Netlogon, Kdc, WinRM, LanmanServer	`C:`

Seuil commun : **15 % d'espace libre minimum** (`group_vars/all.yml`), le même que l'alerte Prometheus. Il a déjà servi : OPS01 était à 60 % sur `/` ; le volume a été étendu à chaud (`lvextend -r -l +100%FREE` , 10 → 18,22 Gio, 33 % utilisés).

Précautions avant tout changement

1. Prévenir le binôme et choisir un créneau.
2. Relever l'état initial : ne jamais maintenir un système déjà en erreur.
3. Snapshot de chaque VM concernée : `virsh snapshot-create-as <VM> maint-AAAA-MM-JJ`.
4. Vérifier l'espace libre (≥ 15 %).
5. Une machine à la fois (`serial: 1`).
6. Ordre de redémarrage : LINUX01, OPS01, **WIN-DC01 en dernier** (annuaire et DNS de tout le labo).
7. Aucun redémarrage implicite.
8. Retour arrière : service → journaux et correction ciblée ; système instable → retour au snapshot.

07

Automatisation Ansible

Critère 3 · 15 pts Automatisation Ansible et reproductibilité

Couvre : inventaire et variables · playbooks Linux et Windows · idempotence · contrôles · commandes

Inventaire et variables

inventory/hosts.yml

```yaml
all:
  children:
    serveurs_linux:
      children:
        linux:
          hosts:
            LINUX01:
              ansible_host: 192.168.50.20
              ansible_user: ansible
              ansible_ssh_private_key_file:
                ~/.ssh/id_ed25519_ansible
        ops:
          hosts:
            OPS01:
              ansible_connection: local
    windows:
      hosts:
        WIN-DC01:
          ansible_host: 192.168.50.10
```

Organisation des variables

```
inventory/
├─ hosts.yml          hôtes, groupes
├─ group_vars/
│  ├─ all.yml         seuil disque 15 %
│  ├─ linux/          sudo, Nginx, ufw
│  ├─ ops/            local, sudo.ws
│  └─ windows/
│     ├─ vars.yml     WinRM 5985, NTLM
│     ├─ ad.yml       OU, groupes, comptes
│     └─ vault.yml    secrets CHIFFRÉS
└─ host_vars/
   └─ LINUX01.yml     disque INFRA-DATA
```

Les variables sont rangées par portée : commune, par groupe, par hôte. Les valeurs sensibles sont isolées dans des `vault.yml` chiffrés.

Connexions : différences Linux et Windows

Cible	Transport	Compte	Élévation
LINUX01	SSH 22, clé ed25519	`ansible`	sudo avec mot de passe (Vault), via `sudo.ws` (E-17)
OPS01	locale (pas de SSH)	`keo`	sudo avec mot de passe saisi (`-K`)
WIN-DC01	WinRM HTTP 5985, NTLM	`svc-ansible@infraready.test`	compte du domaine ; modules `ansible.windows` , `microsoft.ad`

Prérequis sur OPS01 : `ansible` (core 2.20 et collections du paquet), `python3-winrm` , clé `~/.ssh/id_ed25519_ansible` (droits 600, phrase de passe) chargée dans `ssh-agent` , mot de passe du coffre dans le gestionnaire du binôme.

Playbooks

Playbook	Cible	Opérations
`nginx.yml`	linux	Installe Nginx, service actif et activé
`stockage.yml`	LINUX01	Garde-fous, GPT, ext4, montage UUID persistant, groupe, droits 2775, test d'écriture réelle
`linux.yml`	LINUX01	Configuration complète (importe `nginx.yml` + `stockage.yml`)

`windows.yml`	WIN-DC01	OU, groupes, comptes de test, partage `Lab` (NTFS + SMB), relecture des droits
`verification.yml`	tous	Contrôles en lecture seule : services, espace disque, montages, HTTP
`maintenance.yml`	tous	Vérification → mises à jour une machine à la fois → vérification
`supervision.yml`	tous	node_exporter, windows_exporter, Prometheus, Grafana provisionné
`securite.yml`	tous	Pare-feu ufw, sudo protégé, durcissement de WIN-DC01, tests des flux

Idempotence et rejeu

Un playbook **décrit un état**, alors qu'un script enchaîne des actions. Rejoué sur un système déjà conforme, il ne doit rien changer : chaque module vérifie l'état avant d'agir. Les commandes qui ne font que lire sont marquées `changed_when: false`.

Playbook	1er passage	2e passage	failed	Preuves
`nginx.yml`	`changed=1`	`changed=0`	0	`S03_ops01_playbook-run1/2.txt`
`windows.yml`	`changed=4`	`changed=0`	0	`S04_ops01_windows-run1/2.txt`
`windows.yml` (partage)	—	`changed=0` (ok=11)	0	`S04_ops01_windows-partage-run2.txt`
`stockage.yml`	corrections (runs 1–4)	`changed=0` (ok=17)	0	`S04_ops01_stockage-run5/6.txt`
`supervision.yml`	corrections (runs 1–3)	`changed=0` (run 6)	0	`S06_ops01_supervision-run4…6.txt`
`securite.yml`	`changed=3` ×2	`changed=0` ×2	0	`S08_ops01_pare-feu-run1/2.txt`

Contrôles après exécution et gestion des erreurs

- Chaque playbook de configuration se termine par des `assert` et des tests réels (écriture, lecture des ACL, requête HTTP) : un état incorrect **fait échouer** l'exécution au lieu de passer inaperçu.
- Le PLAY RECAP doit afficher `failed=0` et `unreachable=0`. Une tâche en échec arrête l'hôte concerné ; avec `serial: 1`, les autres serveurs ne sont pas touchés.
- Les tâches qui manipulent un mot de passe sont en `no_log: true`.
- Diagnostic d'une erreur : relancer avec `-vvv`, isoler la tâche (`--start-at-task`), vérifier la connexion (`ansible <hôte> -m ping` / `-m win_ping`).

Commandes et reproductibilité

Les commandes de déploiement et la procédure en cinq étapes (syntaxe, cibles, simulation, exécution, rejeu) sont au §03 ; les commandes d'exploitation sont regroupées en annexe B.

08

Supervision — Prometheus et Grafana

Critère 5 · 15 pts Supervision et métriques

Couvre : Prometheus · collecteurs Linux et Windows · Grafana · indicateurs · seuil et alerte

Architecture de collecte

Composant	Hôte	Port	Rôle
node_exporter	LINUX01, OPS01	9100	Métriques système Linux (paquet Ubuntu)
windows_exporter 0.31.8	WIN-DC01	9182	Métriques Windows (MSI officiel prometheus-community)
Prometheus	OPS01	9090	Collecte toutes les 15 s, stocke, évalue les règles d'alerte
Grafana	OPS01	3000	Visualise ; source et tableau de bord provisionnés par fichiers

monitoring/prometheus.yml (extrait)

```yaml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
rule_files: [/etc/prometheus/alertes.yml]
scrape_configs:
  - job_name: linux
    static_configs:
      - targets: ['192.168.50.20:9100']
        labels: { hote: LINUX01 }
      - targets: ['localhost:9100']
        labels: { hote: OPS01 }
  - job_name: windows
    static_configs:
      - targets: ['192.168.50.10:9182']
        labels: { hote: WIN-DC01 }
```

Prometheus fonctionne en **pull** : c'est lui qui interroge `GET /metrics` sur chaque exporter, toutes les 15 s. Chaque cible porte une étiquette `hote` (LINUX01, OPS01, WIN-DC01) utilisée dans les légendes.

La métrique `up` vaut 1 si la dernière collecte a réussi, 0 sinon : c'est la base de l'indicateur de disponibilité et de l'alerte `HoteInjoignable`.

Les règles d'alerte sont évaluées toutes les 15 s ; le `for` impose une durée minimale avant le passage en `firing`, ce qui évite d'alerter sur un pic isolé.

monitoring/alertes.yml (extrait, annotations omises)

```yaml
- alert: HoteInjoignable
  expr: up == 0
  for: 1m
- alert: EspaceDisqueFaibleLinux
  expr: (node_filesystem_avail_bytes{fstype=~"ext4|xfs"}
        / node_filesystem_size_bytes{fstype=~"ext4|xfs"}) < 0.15
  for: 5m
- alert: EspaceDisqueFaibleWindows     # même seuil sur C:, windows_logical_disk_*
- alert: ProcesseurSature              # > 90 % pendant 10 min
```

Quatre règles, validées par `promtool check config` et `promtool check rules` à chaque push (§09).

Tableau de bord « InfraReady - Supervision »



Capture 8. – Tableau de bord provisionné : 4 cibles UP, CPU, mémoire, espace disque (seuil 15 %), table des alertes.

Panneau	Requête PromQL (Linux / Windows)
Disponibilité	`up`
Processeur (%)	`100 - avg by (hote) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100` · idem avec `windows_cpu_time_total`
Mémoire (%)	`(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100` · `windows_memory_available_bytes`
Disque libre (%)	`node_filesystem_avail_bytes / node_filesystem_size_bytes * 100` · `windows_logical_disk_free_bytes`
Alertes	`ALERTS{alertstate=~"pending|firing"}`

La source de données Prometheus (`monitoring/grafana/datasource.yml`) et le tableau de bord (`monitoring/dashboards/infraready.json`) sont **provisionnés** : aucun clic manuel, Grafana est reconstruit à l'identique par le playbook. Le mot de passe administrateur vient du coffre.

Démonstration d'un seuil : disque de LINUX01

Le volume `/srv/infraready` a été rempli volontairement au-delà de 85 %, puis vidé.



Capture 9. – 7,68 % libres sur `/srv/infraready` : barre rouge, alerte en attente.

Heure UTC	Événement (23/10)
avant 09:06	1 % utilisé, alerte inactive
09:06:58	`fallocate -l 4300M` : 7,68 % libres
09:07:13	`pending` (+15 s, évaluation suivante)
09:12:13	`firing` (exactement +5 min : `for: 5m`)
09:16:28	fichier supprimé
09:16:43	alerte résolue (+15 s)

Le seuil de 15 % est le même que celui de `verification.yml` : l'alerte et le contrôle de maintenance concordent. Preuves : `S06_demo-seuil.txt` (1 %, 92 %, 1 %), `S06_ops01_alerte-firing.json` (API Prometheus, état `firing`) et `S06_ops01_chronologie-seuil.txt` (heures reconstituées par `scripts/linux/chronologie-alerte.py`).

09

GitHub et intégration continue

Critère 6 · 10 pts GitHub et intégration continue

Couvre : organisation du dépôt · commits · workflow · portée et limites · preuve

Organisation du dépôt (branche `Keo`)

```
infraready/
├─ README.md · CONTRIBUTING.md · ansible.cfg
├─ .github/workflows/validation.yml
├─ .yamllint · collections/requirements.yml
├─ inventory/      hôtes, variables, coffres
├─ playbooks/      8 playbooks
├─ monitoring/     Prometheus, alertes, Grafana
├─ scripts/linux/ · scripts/windows/
├─ infra/libvirt/  réseau virtuel
├─ configs/        netplan de LINUX01, OPS01
└─ docs/           architecture, adressage,
   ├─ seance-01/ · seance-02/   sécurité, …
   ├─ tracabilite.md · livrables/ (Typst)
   ├─ schemas/     schéma draw.io
   └─ preuves/     131 preuves
S<séance>_<machine>_<objet>
```

Versionnement et commits

Tout ce qui configure le labo est versionné : playbooks, inventaire, configurations, scripts, supervision, documentation et preuves. Les commits suivent la convention de `CONTRIBUTING.md` : un type, une portée, une description de l'intention à l'infinitif, en français.

Type	Commits
`feat` (fonctionnalité)	12
`docs`	13
`test` (preuve, démonstration)	7
`fix` (correction)	6
`chore`, `ci`, `style`	5

git log --oneline (extrait)

```
e25bddf ci: contrôler l'inventaire et les scripts shell, Python et PowerShell
b49ec4c docs(preuves): journaux Windows, console AD, exécutions GitHub Actions et matériel de WIN-DC01
8376f72 ci: valider le dépôt avec yamllint, ansible-lint, syntax-check, promtool et gitleaks
c95cb74 fix(securite): exécuter les contrôles sans élévation (ansible_become prime sur become)
5e6edd2 test(securite): appliquer le pare-feu ufw, prouver les flux autorisés et refusés, et l'idempotence
07f52d4 test(supervision): démontrer l'alerte de seuil disque (chronologie pending/firing/résolue)
2940d07 docs(incident): diagnostiquer et documenter l'absence de métriques de LINUX01 (pare-feu port 9100)
a4d23db feat(supervision): ajouter Prometheus, Grafana, node_exporter et windows_exporter
```

Workflow GitHub Actions `validation.yml`

Déclenché à chaque `push` et `pull_request`, sur un runner `ubuntu-latest` hébergé par GitHub, avec des droits en lecture seule (`permissions: contents: read`). Les collections sont installées depuis `collections/requirements.yml`.

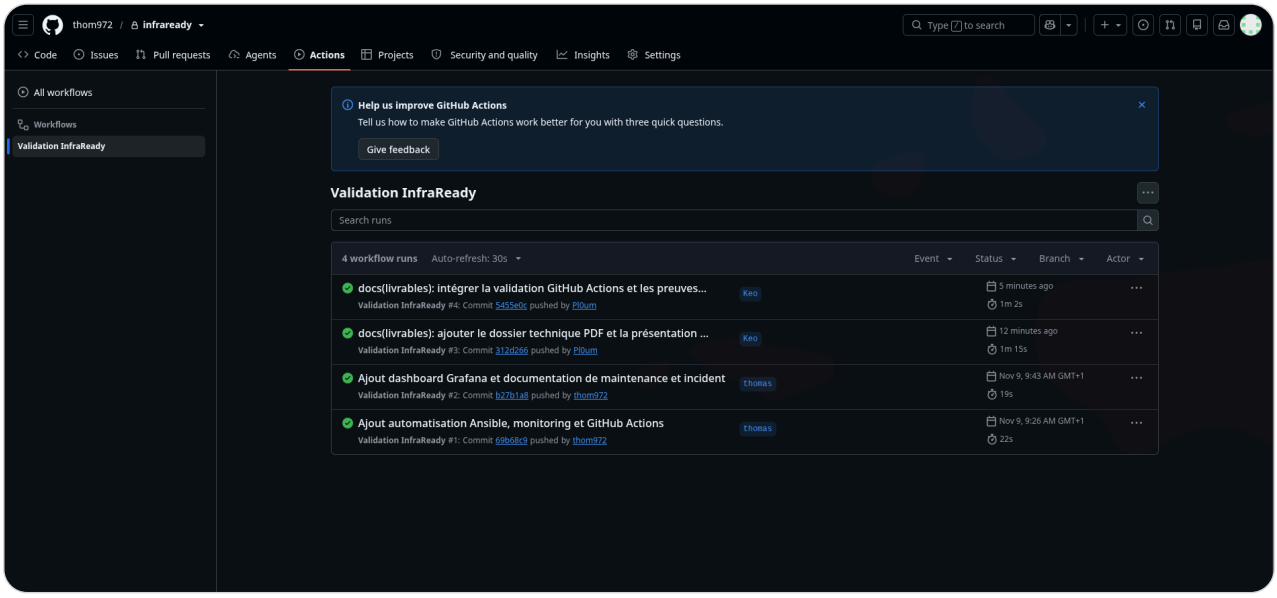
Étape	Contrôle	Exécution du 10/11
gitleaks	Recherche de secrets dans **tout l'historique Git**	0 fuite
Coffres	Chaque `vault.yml` commence par `$ANSIBLE_VAULT;1.1;AES256`	3/3
yamllint	`yamllint --strict .` avec `.yamllint` compatible ansible-lint	0 alerte
ansible-lint	Bonnes pratiques Ansible ; remarques corrigées sans changer le comportement	profil production
Inventaire	`ansible-inventory --graph` : groupes et hôtes lisibles	3 hôtes
Playbooks	`ansible-playbook --syntax-check` sur les 8 playbooks	8/8

Scripts	`shellcheck` (Bash), `py_compile` (Python), PSScriptAnalyzer (PowerShell, erreurs bloquantes)	0 erreur
promtool	`check config` de `prometheus.yml` et `check rules` de `alertes.yml`	4 règles

Chaque étape échoue au moindre problème et bloque l'exécution : une modification qui casse un playbook, une règle d'alerte ou un script, ou qui contient un secret, apparaît en rouge sur le commit concerné.

Portée du pipeline
Le workflow **valide le dépôt**, il ne **déploie** rien. Le runner hébergé par GitHub est une machine d'Internet : il n'a aucune route vers 192.168.50.0/24, réseau privé NATé derrière le poste hôte, et il ne doit pas en avoir (il faudrait exposer WinRM et SSH, ou y placer des secrets). Le déploiement reste manuel depuis OPS01. L'automatiser demanderait un runner auto-hébergé **dans** le labo.

Preuve de réussite
Exécution `42786343692` du 10/11/2026, déclenchée par un `push` sur la branche `Keo` : toutes les étapes réussies en 1 min 14 s, 46 commits analysés par gitleaks (« no leaks found »), ansible-lint « 0 failure(s), 0 warning(s) » en profil `production`, 8 playbooks, configuration et 4 règles Prometheus valides. Sortie : `S08_github_actions-validation-scripts.txt` (exécution précédente : `S08_github_actions-validation.txt`).



Capture 10. – Onglet **Actions** du dépôt : exécutions réussies du workflow sur les branches `Keo` et `thomas`.

10

Sécurité et gestion des accès

Critère 7 · 5 pts Sécurité et gestion des accès

Couvre : moindre privilège et ports · secrets · sécurité des opérations · périmètre

Protection des secrets

- Mots de passe uniquement dans le gestionnaire du binôme et dans trois **Ansible Vault** chiffrés en AES256 (`linux` , `ops` , `windows`), saisis à l'exécution par `--ask-vault-pass` .
- `no_log: true` sur toute tâche qui manipule un mot de passe ; le mot de passe Grafana est protégé dans la ligne de commande (filtre `quote`).
- `.gitignore` bloque ISO, disques, clés ; `git diff --staged` relu avant chaque commit ; la CI refuse un coffre non chiffré et gitleaks ne trouve aucun secret dans l'historique.
- Clés SSH privées uniquement sur le poste qui les utilise, droits 600, avec phrase de passe.

Pare-feu ufw des serveurs Linux (`playbooks/securite.yml`)

Refus par défaut en entrée ; chaque flux autorisé a une source précise. Les règles sont des variables versionnées (`group_vars/*/vars.yml`).

Hôte	Port	Source	Motif
LINUX01	22	.1 · .30	SSH admin · Ansible
LINUX01	80	labo /24	Nginx
LINUX01	9100	.30 seul	collecte Prometheus
OPS01	22	.1	SSH admin
OPS01	3000 · 9090	.1	Grafana · Prometheus
WIN-DC01	5985 · 9182	.30 seul	WinRM · collecte
WIN-DC01	53 · 445	labo /24	DNS · SMB

S08_hote_tests-pare-feu.txt — depuis l'hôte

```
== SSH (autorisés)
linux01
ops01
== Grafana depuis l'hôte (autorisé)
200
== node_exporter LINUX01 depuis l'hôte
   (doit être REFUSÉ)
curl: (28) Connection timed out
     after 5000 milliseconds
```

L'ordre des tâches est volontaire : les flux nécessaires sont autorisés **avant** l'activation du pare-feu, pour ne jamais couper la session SSH d'Ansible. Le playbook se termine en vérifiant que les flux autorisés fonctionnent toujours et que les autres sont refusés.

Moindre privilège

Mesure	Détail	Écart
sudo protégé	Le compte `ansible` passe de `NOPASSWD` à sudo **avec** mot de passe (Vault) ; règle validée par `visudo -cf` ; SSH par clé uniquement	E-14 corrigé
Compte de service	`svc-ansible` reste Admin du domaine (**risque accepté**) : non délégable, WinRM depuis OPS01 seulement, mot de passe dans le coffre	E-15
Droits par groupe	AD : groupes globaux ; Linux : groupe `infraready` , setgid, pas de 777	—

Pas de RDP	Console SPICE suffisante : un port d'administration de moins	E-09
WinRM	Ni `TrustedHosts = *`, ni Basic, ni `AllowUnencrypted` ; NTLM chiffre les échanges	E-08

Sécurité des opérations et périmètre

- **Partitionnement et formatage** : cible désignée par son numéro de série, `assert` qui refuse le disque système, `force: false` (§06).
- **Changements majeurs** : snapshot préalable, simulation `--check --diff`, une machine à la fois, retour arrière documenté (`sudo ufw disable` depuis la console de la VM).
- **Rejeu** : idempotence prouvée, donc rejouer un playbook est sans risque.
- **Périmètre** : uniquement les trois VM du réseau 192.168.50.0/24 ; l'incident (§11) a été provoqué volontairement, de façon réversible et documentée. Interdits écrits : désactiver globalement un pare-feu, créer une règle « tout autoriser », scanner hors du labo.

11

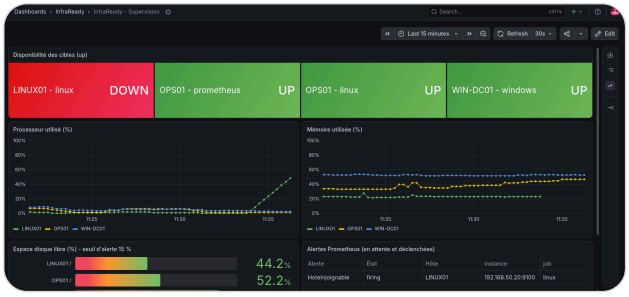
Rapport d'incident — absence de métriques de LINUX01

Critère 8 · 10 pts Documentation, preuves et diagnostic

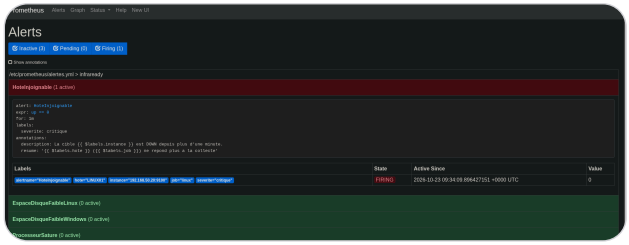
Couvre : méthode de diagnostic · correction et retour à la normale · preuves

Nature	Incident **provoqué volontairement** le 23/10/2026, réversible : table nftables `incident` bloquant le port 9100 de LINUX01. Diagnostic mené comme si la cause était inconnue.
Impact	Plus aucune métrique de LINUX01 pendant 4 min 30 s ; SSH et Nginx restent disponibles.
Détection	Automatique : cible `DOWN` , puis alerte `HoteInjoignable` .
Durée	09:34:03 → 09:38:33 UTC ; alerte résolue à 09:38:48.

1. Symptôme



Capture 11. – Grafana : LINUX01 DOWN, alerte HoteInjoignable en firing.



Capture 12. – Prometheus : alerte FIRING (`up == 0` , `for: 1m`).

Erreur relevée par Prometheus :
`Get "http://192.168.50.20:9100/metrics": context deadline exceeded` .

2. Tests et journaux, du réseau vers le service

#	Test	Résultat	Conclusion
1	`ping -c 2 192.168.50.20` (OPS01)	2/2 reçus	VM allumée, réseau OK
2	`curl -m 5 …:9100/metrics` (OPS01)	délai dépassé (code 28)	le **port** ne répond pas
3	API `/api/v1/targets`	`context deadline exceeded`	délai, et non refus
4	`systemctl is-active prometheus-node-exporter`	`active`	service non arrêté
5	`ss -lnt | grep 9100`	`LISTEN *:9100`	le service écoute
6	`curl -m 5 localhost:9100/metrics`	délai dépassé	filtrage **sur la machine**
7	`journalctl -u prometheus-node-exporter`	aucune erreur pendant l'incident	service sain
8	`nft list ruleset`	`tcp dport 9100 drop`	**cause confirmée**

Hypothèse	Tests	Verdict
VM éteinte / réseau coupé	1	Écartée
node_exporter arrêté	4, 5	Écartée
Service en erreur	7	Écartée
Filtrage sur LINUX01	2, 3, 6, 8	Confirmée

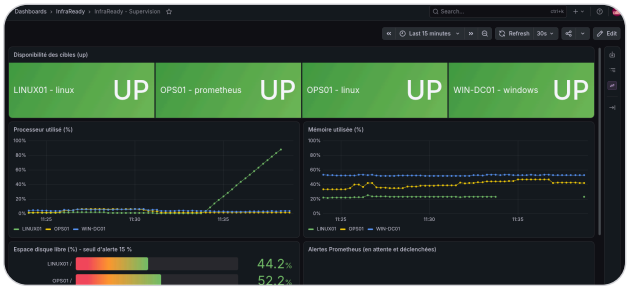
Le raisonnement clé. Un délai dépassé, et non un refus de connexion, oriente vers un filtrage : une règle `drop` ignore les paquets sans répondre, alors qu'un service arrêté provoque un refus immédiat. Le test local échouant lui aussi, le filtrage était sur la machine elle-même et non sur le chemin réseau.

S07_linux01_diagnostic.txt

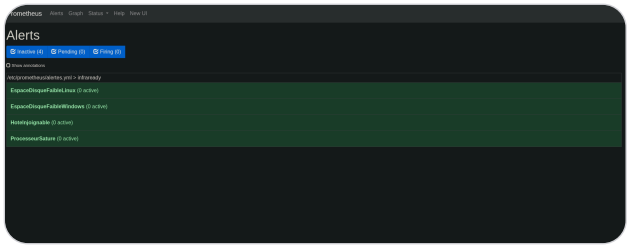
```
== Service             active
== Écoute locale       LISTEN 0 4096 *:9100 *:*
== Pare-feu
table inet incident {
    chain input { type filter hook input priority filter; policy accept;
        tcp dport 9100 drop }
}
```

3. Correction et retour à la normale

```
sudo nft delete table inet incident     # seule la table fautive est supprimée
curl -s http://192.168.50.20:9100/metrics | head -1     # depuis OPS01
```



Capture 13. – Retour à la normale : 4 cibles UP.



Capture 14. – Prometheus : 4 règles inactives.

Première collecte réussie après 09:38:33, alerte résolue à 09:38:48 (évaluation suivante). Preuves : `S07_ops01_diagnostic.txt` (tests 1 à 3), `S07_ops01_retour-normale.txt`, chronologie `S07_ops01_chronologie-incident.txt`.

Enseignements

1. **La supervision a détecté seule l'incident**, dans le délai prévu : cible `DOWN` en 15 s, alerte `firing` en 1 min 15 s.
2. **Les contrôles locaux ne suffisent pas** : `verification.yml` s'exécute sur la machine et ne voit pas un filtrage entre Prometheus et sa cible. Les deux approches sont complémentaires.
3. **Une règle de pare-feu doit être versionnée** : c'est fait depuis la séance 8 avec `securite.yml` (ufw géré par Ansible).
4. **Codes de retour dans un pipeline** : `curl … | head -1; echo $?` affiche le code de `head` (0), pas celui de `curl` (28) ; utiliser `${PIPESTATUS[0]}`.

12

Écarts, choix et évolutions

Critère 8 · 10 pts Documentation, preuves et diagnostic

Couvre : limites annoncées honnêtement · cohérence des livrables

Écarts au kit et au sujet

ID	Écart	Justification
E-01/02	QEMU/KVM + virt-manager sur Arch Linux, au lieu de VMware	Hyperviseur natif de l'hôte, libre, snapshots qcow2
E-03/04	Réseau libvirt NAT ; DNS temporaire .1 avant AD DS	« Chemin NAT contrôlé » du kit ; DNS fonctionnel jusqu'à la promotion
E-05	Noms d'hôte système en minuscules	Convention Linux ; DNS et Ansible ignorent la casse
E-06	WIN-DC01 en disque SATA et carte e1000e, au lieu de virtio	Pilotes intégrés à l'installeur Windows (`S08_hote_win-dc01-materiel.txt`)
E-08/09	WinRM 5985 limité à OPS01 ; RDP désactivé	Transport Windows exigé par le sujet ; surface réduite
E-11	Inventaire YAML	Proposé par le sujet
E-12	Ubuntu Server 26.04.1 au lieu de 24.04	Version installée ; impose E-17
E-14	sudo sans mot de passe pour `ansible` (séances 3 à 7)	**Résolu en séance 8** : sudo avec mot de passe (coffre), SSH par clé
E-15	`svc-ansible` Admin du domaine	**Risque accepté** ; compensations : non délégable, WinRM limité à OPS01, NTLM, coffre
E-16	Comptes de test sans changement de mot de passe imposé	Tests reproductibles, playbook idempotent
E-17	Ansible utilise `sudo.ws` au lieu de sudo-rs, sur OPS01 puis LINUX01	sudo-rs modifie l'invite attendue par Ansible

Choix assumés pour le laboratoire

Choix	Statut	Évolution en production
`svc-ansible` Admin du domaine (E-15)	Risque accepté	Délégation limitée à `OU=InfraReady`
Alertes visibles dans Prometheus et Grafana, sans notification (pas d'Alertmanager)	Assumé	Alertmanager + courriel
Paquet Ubuntu de Prometheus : interface classique seulement	Assumé	Consultation dans Grafana
Mises à jour Windows installées manuellement	Choix	Le redémarrage du DC reste une décision humaine

Bilan

Les huit parties du sujet sont réalisées et prouvées : trois VM et un domaine fonctionnel, huit playbooks idempotents, une supervision multiplateforme qui a détecté seule un incident, un pare-feu versionné, et 14 contrôles GitHub Actions à chaque push et pull request.

Annexe A — Correspondance avec les grilles

Grille des pairs (soutenance, /100) et grille du formateur (dépôt et livrables, /100). Pour chaque sous-critère : où se trouve la réponse, et l'état de la preuve.

Sous-critère	Pts	§	Preuve / ce qu'on montre	État
1. Infrastructure, réseau et DNS — 15 pts				
Architecture et rôle des machines	3	02	Schéma, tableau des rôles et ressources	Validé
Plan d'adressage IP	3	02	Plan, DHCP hors adresses fixes, netplan	Validé
Réseau virtuel et connectivité	3	02	XML libvirt, `S02_*_reception.txt` (ping)	Validé
DNS et domaine	3	02	Zone, `S04_dns-clients-linux.txt`, lecture d'un échec	Validé
Schéma et cohérence	3	02	Schéma conforme aux IP, ports et pare-feu réels	Validé
2. Administration Windows et Linux — 20 pts				
Active Directory et DNS Windows	4	04	Promotion, console AD : OU, 2 groupes, 4 comptes	Validé
Comptes, groupes et droits	4	04–05	Partage NTFS/SMB, test refusé, setgid 2775	Validé
Services et processus Linux	3	05	Nginx : systemctl, ps, journalctl, check-nginx.sh	Validé
Stockage, permissions, FS	3	05–06	findmnt, fstab UUID, droits testés	Validé
Journaux et diagnostic	3	05, 04	journalctl (3 preuves) ; journaux Système et Directory Service	Validé
Tests et validation	3	04–06	Écriture réelle plutôt que `test -w`, smbclient	Validé
3. Ansible et reproductibilité — 15 pts				
Inventaire et variables	3	07	hosts.yml, group_vars, host_vars, vault	Validé
Playbooks Linux et Windows	3	07	8 playbooks, SSH et WinRM/NTLM	Validé
Pertinence et lisibilité	2	07	Tâches nommées, garde-fous en tête	Validé
Idempotence et rejeu	3	07	5 couples run 1 / run 2 `changed=0`	Validé
Contrôles après exécution	2	07	`assert`, tests réels, PLAY RECAP	Validé
Commandes et reproductibilité	2	07, B	Procédure en 5 étapes, prérequis	Validé
4. Stockage et maintenance — 10 pts				
Stockage et montage	3	06	INFRA-DATA, ext4, UUID, `nofail`, test après reboot	Validé
Maintenance Linux et Windows	3	06	`maintenance.yml`, 4 KB, procédure DC	Validé
Contrôle de l'état	2	06	`verification.yml`, seuil 15 %, extension LVM	Validé
Précautions avant changement	2	06, 10	Snapshots, ordre de redémarrage, `serial: 1`	Validé
5. Supervision et métriques — 15 pts				
Rôle de Prometheus	3	08	Modèle pull, `prometheus.yml`, 15 s	Validé

Collecte Linux et Windows	4	08	node_exporter ×2, windows_exporter, 4 UP	Validé
Grafana et source	2	08	Datasource et tableau de bord provisionnés	Validé
Indicateurs	3	08	CPU, mémoire, disque, `up`, PromQL	Validé
Seuil, alerte, interprétation	3	08, 11	Disque pending → firing → résolue ; action	Validé
6. GitHub et intégration continue — 10 pts				
Organisation du dépôt	2	09	Arborescence commentée	Validé
Versionnement et commits	2	09	43 commits conventionnels	Validé
Workflow GitHub Actions	3	09	`validation.yml` : 14 contrôles, du secret au script	Validé
Portée et limites	2	09	Valide sans déployer ; runner sans accès au /24	Validé
Preuve de réussite	1	09	Exécution 42786343692, onglet Actions	Validé
7. Sécurité et gestion des accès — 5 pts				
Moindre privilège et accès	2	10	Matrice ufw, sudo protégé, tests refus	Validé
Protection des secrets	1	10	3 coffres AES256, `no_log`, contrôle CI	Validé
Sécurité des opérations	1	06, 10	Garde-fous disque, snapshots, `--check`	Validé
Périmètre et limites	1	01, 10	Labo /24 uniquement, incident réversible	Validé
8. Documentation, preuves et diagnostic — 10 pts				
Dossier technique et architecture	2	01–02	Ce dossier	Validé
Preuves de fonctionnement	2	toutes, C	131 preuves nommées par séance	Validé
Méthode de diagnostic	2	11	8 tests ordonnés, hypothèses discriminées	Validé
Correction et retour à la normale	2	11	Commande ciblée, captures 4 UP	Validé
Limites et communication	2	12	Écarts justifiés, choix assumés	Validé

Points spécifiques à la grille du formateur

Playbook Windows : prérequis de connexion précisés (§07). **Mises à jour Windows** : procédure documentée **et testée** (§06). **Cohérence du dépôt (1 pt)** : liens et noms de preuves vérifiés le 10/11 ; chaque preuve citée existe dans `docs/preuves/`. **Dossier PDF relié au dépôt** : ce PDF est `docs/dossier-technique.pdf`, ses sources sont dans `docs/livrables/`.

Annexe B — Commandes de référence

Commandes extraites de la documentation du dépôt (fichier source indiqué en titre).

Réseau virtuel — `infra/libvirt/`

```
sudo virsh net-define infra/libvirt/infraready-
network.xml
sudo virsh net-start infraready
sudo virsh net-autostart infraready
```

Réseau des VM — `configs/*/netplan/`

```
sudo netplan generate && sudo netplan try
```

Stockage — `docs/stockage.md`

```
findmnt /srv/infraready
sudo findmnt --verify      # puis redémarrage de
LINUX01
```

Snapshots — `docs/maintenance.md`

```
virsh snapshot-create-as <VM> maint-AAAA-MM-JJ \
  "Avant maintenance"
```

Coffres — `docs/ansible.md`

```
EDITOR=nano ansible-vault edit \
  inventory/group_vars/linux/vault.yml
ansible-vault view inventory/group_vars/ops/
vault.yml
head -1 inventory/group_vars/*/vault.yml
```

Supervision — `docs/supervision.md`

```
ssh linux01 'df -h /srv/infraready'
ssh linux01 'fallocate -l 4300M \
  /srv/infraready/demo-seuil.bin'
ssh linux01 'rm /srv/infraready/demo-seuil.bin'
```

Incident — `docs/incident.md`

```
ping -c 2 192.168.50.20                # OPS01
curl -m 5 http://192.168.50.20:9100/metrics
systemctl is-active prometheus-node-exporter
ss -lnt | grep 9100
curl -m 5 localhost:9100/metrics
journalctl -u prometheus-node-exporter
sudo nft list ruleset
sudo nft delete table inet incident
```

Windows — `docs/maintenance.md`

```
Get-HotFix |
  Sort-Object InstalledOn -Descending |
  Select-Object -First 5
dcdiag /test:Services /test:Advertising `
  /test:NetLogons /test:SysVolCheck
Get-WinEvent -LogName System -MaxEvents 50 |
  Where-Object Level -le 3
```

Validation locale — `README.md`

```
yamllint --strict .
ansible-lint
shellcheck scripts/linux/*.sh
```

Annexe C — Index des preuves

Toutes les preuves sont dans `docs/preuves/`, nommées `S<séance>_<machine>_<objet>`. L'index complet des 131 preuves est `docs/preuves/README.md` ; la correspondance exigence par exigence est `docs/tracabilite.md`. Sélection des preuves citées dans ce dossier :

Fichier	Ce qu'il démontre
`S02_*_reception.txt` (×4)	Réception des VM et de l'hôte : IP, route, DNS, ping, versions
`S02_winrm-ops01-ok-linux01-refuse.png`	WinRM limité à OPS01
`S03_ops01_playbook-run1/2.txt`	Idempotence du playbook Nginx
`S03_linux01_check-nginx-avant/apres.txt`	Service applicatif installé et actif
`S04_win-dc01_ad-structure.txt`, `_zone-dns.txt`	Annuaire, groupes, zone DNS
`S04_win-dc01_aduc-groupes.png`, `_aduc-utilisateurs.png`	Console AD : OU, groupes et comptes
`S04_dns-clients-linux.txt`	Résolution DNS depuis les clients Linux
`S04_linux01_test-partage.txt`	Droits du partage : autorisé, refusé
`S04_linux01_stockage-verif.txt`, `_disque-identification.txt`	Montage persistant, identification de la cible
`S04_ops01_stockage-run1…6.txt`, `_windows-run1/2.txt`	Corrections successives puis idempotence
`S05_ops01_maintenance*.txt`, `_verification.txt`	Maintenance et contrôles des 3 hôtes
`S05_win-dc01_historique-wu.png`	Mises à jour Windows installées
`S06_demo-seuil.txt`, `S06_grafana-*.png`, `S06_prometheus-*.png`	Supervision, cibles, seuil disque
`S06_ops01_alerte-firing.json`, `_chronologie-seuil.txt`	Seuil disque : état firing et chronologie
`S07_*.txt`, `S07_grafana-*.png`, `S07_prometheus-*.png`	Incident : état initial, diagnostic, chronologie, retour à la normale
`S08_ops01_pare-feu-run1/2.txt`, `_simulation.txt`	Pare-feu : simulation, application, idempotence
`S08_hote_tests-pare-feu.txt`	Flux autorisés et refusés
`S08_ops01_durcissement-*.txt`, `_controle-sudo-manuel.txt`	Sécurité complète : sudo protégé, durcissement de WIN-DC01
`S08_win-dc01_journal-systeme.png`, `_journal-directory-service.png`	Journaux Windows relus
`S08_hote_win-dc01-materiel.txt`	Matériel virtuel réel de WIN-DC01 (E-06)
`S08_github_actions-validation*.txt`, `_executions.png`	Exécutions réussies du workflow GitHub Actions